



MediComm

PRODUCT, TECHNOLOGY & PITCH GUIDE

What is behind the website.

A clear, evidence-based walkthrough of the product, user experience, content engine, architecture, business model, risks, and pitch story.

Prepared

18 August 2026

Stage

Working product / MVP

Audience

Founders, partners, investors &
technical reviewers



MediComm in one page

MediComm is a focused digital workspace for medical students preparing for high-stakes exams. It combines question-led learning with performance feedback, peer communities, rankings, and short competitive battles.

“Practice the right medical questions, understand weak areas, and keep momentum—inside one connected learning workspace.”

19

medical subjects in the official catalogue

15

exam-year collections represented

13,035

distinct question records served after de-duplication

10

main product destinations

The problem

- ! Exam preparation is fragmented across question banks, notes, analytics, chats, and peer groups.
- ! Students often know their score but not what to study next.
- ! Consistency is hard when feedback loops are slow and solitary.

The product answer

- ✓ Structured PYQ, AI-supplemental, and USMLE-style practice.
- ✓ Confidence, accuracy, pacing, flags, and weak-topic signals.
- ✓ Community, direct messages, leaderboards, and 1v1 duels.

Pitch discipline: these figures describe the current repository and runtime inventory—not verified active users, revenue, or clinical outcomes.

One workspace, four reinforcing loops

1. Learn

Choose an exam, year, subject, chapter, or topic. Solve MCQs with clinical images, laboratory findings, explanations, flags, confidence, and keyboard shortcuts.

IMPLEMENTED

2. Understand

Turn attempts into accuracy, pace, study-time, subject-mastery, trend, and “needs attention” views.

IMPLEMENTED LOCALLY

3. Connect

Discover or create communities, publish threads with images, reply, manage members, find peers, and exchange direct messages.

IMPLEMENTED

4. Compete

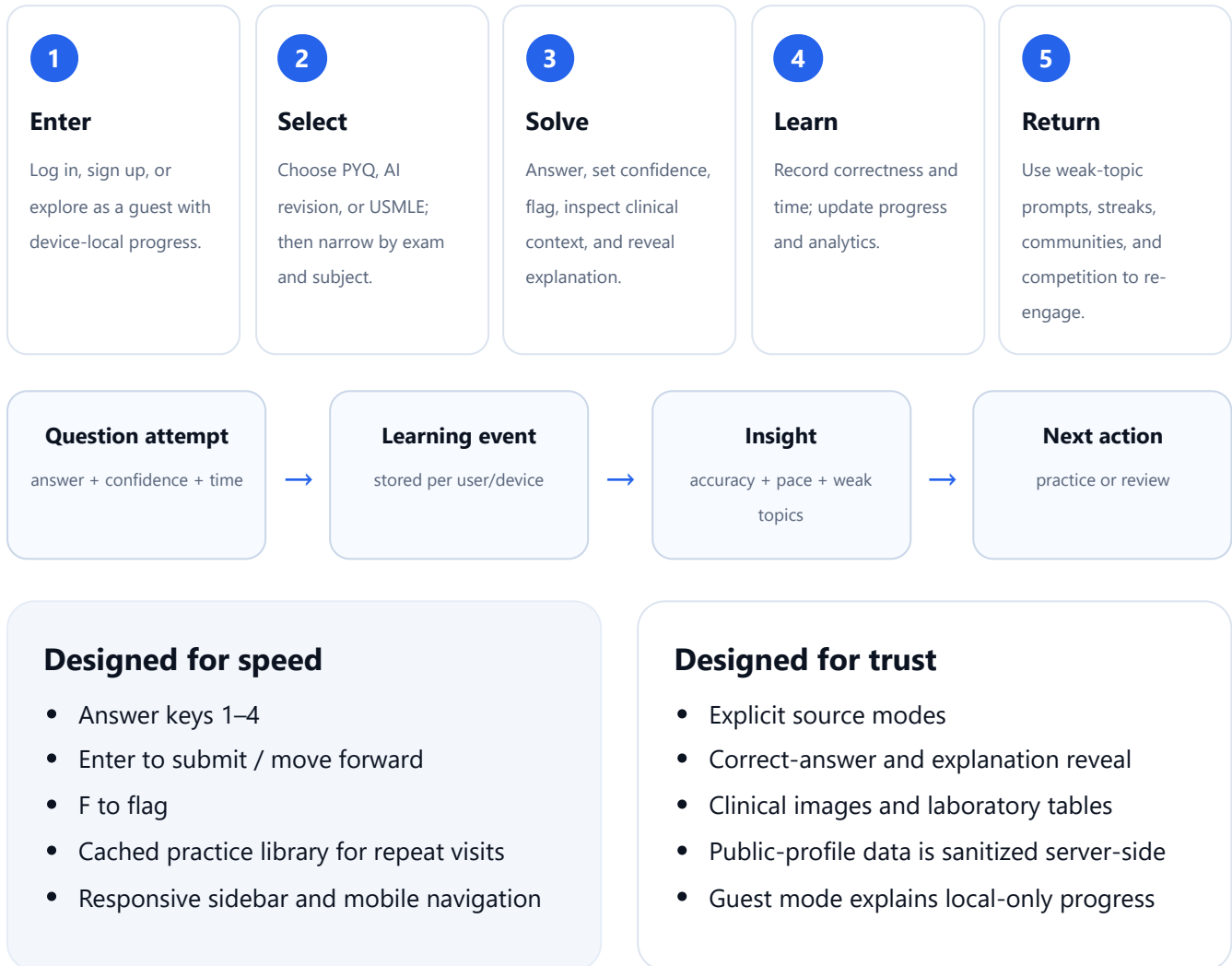
Use rankings, regional/college filters, bot practice, or rated matchmaking with Elo-style rating changes.

IMPLEMENTED

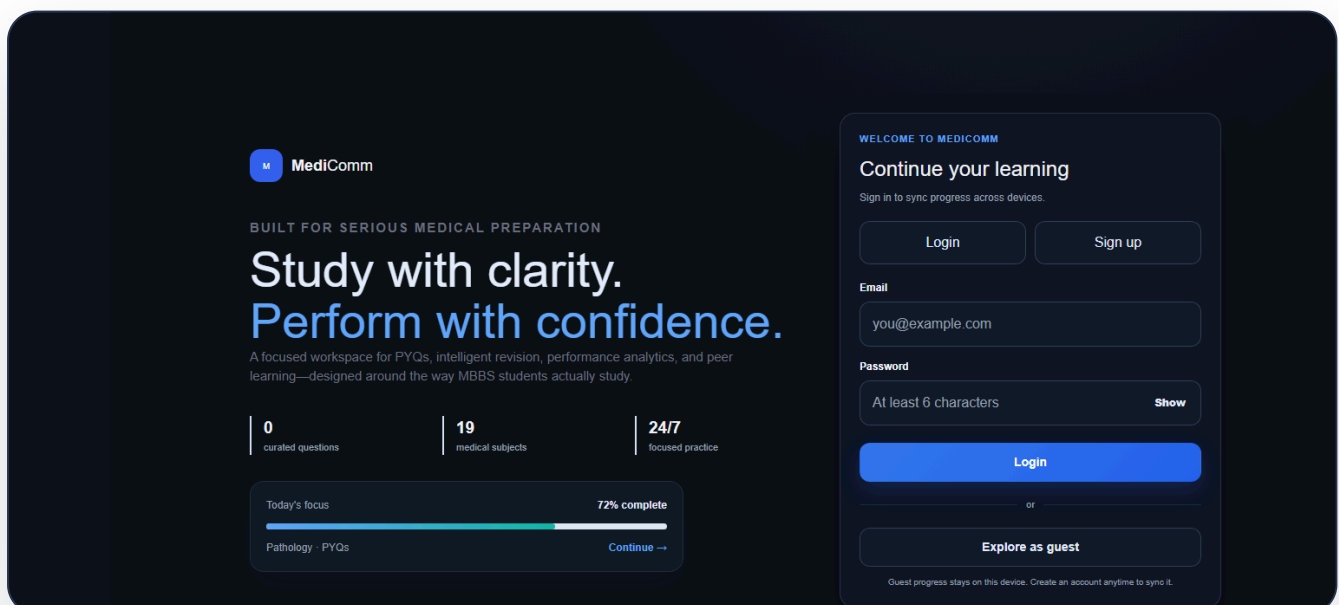
Main navigation

DESTINATION	JOB FOR THE STUDENT	CURRENT STATE
Home / Dashboard	Orient, continue studying, see goals and activity.	WORKING
Practice	Choose and solve question sets across three content modes.	WORKING
Analytics	Review accuracy, pace, study time, mastery, and weak areas.	WORKING
Leaderboard	Compare ratings by state and college.	WORKING
Communities	Discuss, reply, share images, manage groups, and message peers.	WORKING
Compete	Enter bot or rated five-question, three-minute duels.	WORKING
Pricing	Understand Lite, Ultra, and Premium positioning.	UI ONLY
Profile / Settings	Manage identity, theme, study preferences, and account.	MIXED

The core student journey



Current authentication surface



The product's content backbone

3,320

official-tagged records

9,695

unique AI-supplemental records

20

USMLE-tagged records

Together, the API can assemble **13,035 distinct question records**. Counts were computed from [data/practice-question-bank.json](#) plus [runtime-data/users.json](#), de-duplicated by question ID. "Official-tagged" describes the source field in the repository; independent rights and editorial verification remain a business responsibility.

Official catalogue coverage

SUBJECT	RECORDS
General Medicine	857
Ophthalmology	500
Pathology	421
Pharmacology	407
Microbiology	370
Community Medicine	356
Anatomy	276
Biochemistry	266
Forensic Medicine	210
Physiology	177

SUBJECT	RECORDS
ENT	139
Anesthesia	103
General Surgery	96
Obstetrics & Gynaecology	93
Pediatrics	42
Orthopedics	26
Dermatology	22
Psychiatry	20
Radiology	19
Total	4,400*

*The catalogue total includes 1,080 records tagged as AI within the catalogue file; hence the 3,320 official-tagged total above.

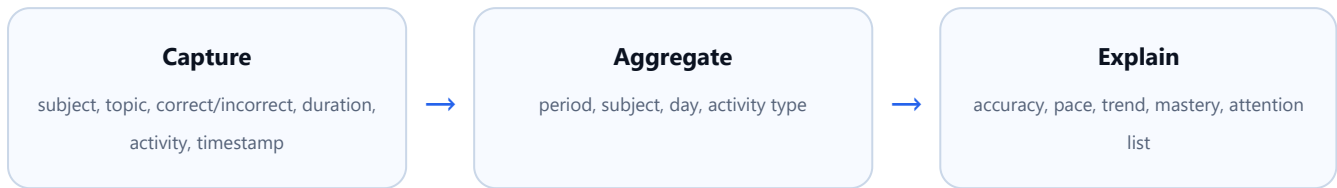
Question schema

Exam, year, subject, topic, chapter, prompt, lead-in, four options, answer index, explanation, difficulty, tags, source, images, lab findings, and source provenance fields.

Study controls

Mode, exam/year, subject, topic/chapter, randomization, answer submission, explanation, confidence, flagging, progress, and next-question navigation.

How attempts become useful feedback



Analytics currently computed

- ✓ Accuracy over 30 / 90 / 365 days
- ✓ Questions solved
- ✓ Average time per question
- ✓ Total measured study time
- ✓ Daily accuracy trend
- ✓ Subject mastery
- ✓ Time by PYQ / revision / battle
- ✓ Top weak topics from incorrect answers

Important implementation fact

Detailed analytics events and practice progress are stored in browser `localStorage`, scoped to the current user identifier and capped at the most recent 2,000 events.

Meaning: it works well for a prototype and a single device, but full cross-device learning history requires moving event storage to the server/database.

PROTOTYPE ARCHITECTURE

Learning loop as a pitchable moat

CONTENT DEPTH

More structured attempts create more meaningful learner signals.

PERSONALIZATION

Signals can prioritize the next set by weakness, confidence, recency, and pace.

RETENTION

Visible improvement, social accountability, and short competition can reinforce return behavior.

What is not yet present: a server-side recommendation model, validated adaptive algorithm, spaced-repetition scheduler, or outcomes study. Present those as roadmap, not current capability.

Why MediComm is more than a question bank

Communities and direct messages

- Create and join topic communities
- Post text threads up to 300 words
- Attach images up to 5 MB
- Reply to threads and expand conversations
- Admin member removal
- Search users and open private conversations
- Periodic refresh for new messages

Leaderboards and 1v1 duels

- Global list with state and medical-college filtering
- Rating and streak display
- Bot mode for instant play
- Rated queue using server-side session records
- Five questions in three minutes
- Elo-style rating update with K-factor 32
- Result history and profile linkage

The engagement thesis

1

Individual value

A student gets immediate practice value alone.

2

Identity

Profile, college, state, rating, and streak create context.

3

Peer value

Communities and messages add support and knowledge exchange.

4

Competition

Duels and rank turn recall into a short repeatable format.

5

Return loop

Activity and social feedback create another reason to come back.

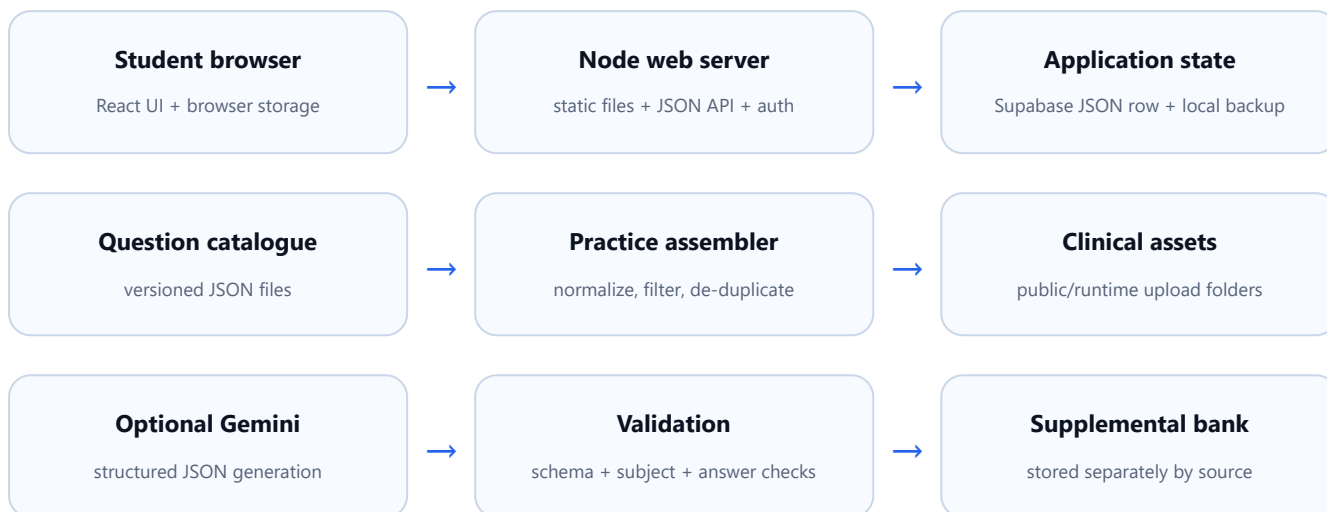
What is real today

Server routes, persisted records, sanitization, queue operations, community membership, direct conversations, duel completion, and rating updates.

What needs scale work

WebSockets/realtime events, moderation tooling, reporting, abuse prevention, queue concurrency controls, notification delivery, and normalized database tables.

How the website works



Architecture layers

Experience	React screens, shell navigation, responsive CSS, accessible Radix primitives, Lucide icons, motion.
Client state	React hooks for live UI; <code>localStorage</code> for token, theme, cache, progress, analytics, and read notifications.
API	Node native HTTP routing for authentication, profiles, content, community, messages, leaderboard, duels, and AI generation.
Persistence	Supabase REST against one <code>app_state</code> JSONB row; local JSON is seed, backup, and fallback.
Delivery	Vite production build served by Node; Render blueprint targets Node 22.

Plain-English summary: one deployable service hosts both the website and backend. This keeps the MVP simple; the next scale step is to separate durable data domains, not necessarily to split into microservices.

What the user-facing website is made with

Core

React 18.3

Vite 5.4

JavaScript / JSX

CSS

The application lives primarily in `src/App.jsx`, with a reusable shell and small UI primitives.

Interface libraries

Radix UI

Lucide React

Framer Motion

Tailwind utilities installed

Radix provides dialog, dropdown, progress, switch, tabs, and tooltip foundations.

Client responsibilities

AREA	WHAT HAPPENS IN THE BROWSER
Navigation	View switching without a URL router; animated page transitions and command palette.
Practice	Catalogue selection, question state, keyboard actions, progress, flags, confidence, and attempt timing.
Analytics	Local event aggregation into KPIs, trend chart, mastery, activity mix, and weak-topic list.
Competition	Timer, answer state, bot-opponent simulation, queue polling, and result display.
Personalization	Dark/light theme, per-user browser cache, notification-read state, and guest progress.

Strengths

- ✓ Single coherent workspace
- ✓ Responsive desktop/mobile shell
- ✓ Clear loading and guest states
- ✓ Accessible component foundations
- ✓ Fast static build and browser caching

Engineering debt

- ! `App.jsx` is more than 4,000 lines
- ! No URL-level routing/deep links
- ! Large practice payload
- ! Some installed dependencies are not yet used
- ! Several settings and pricing controls are presentation-only

The application server

The backend is a single Node.js 22 service built with the native HTTP module—no Express framework. It serves the compiled website, JSON endpoints, and uploaded files.

IDENTITY

Signup, login, session validation, logout, profile updates, public profiles, and user search.

LEARNING

Practice library, filterable questions, AI generation, profile statistics, platform summary, leaderboard.

NETWORK

Communities, membership, threads, images, direct conversations, rated queue, duel completion.

Request pattern



Useful implementation choices

- Server-generated random IDs and session tokens
- Timing-safe password comparison
- Output sanitizers for public/private data
- Deterministic seeded duel question selection
- Immutable caching for built assets
- Local fallback if Supabase is unavailable

Scale limitations

- Whole application state is cloned and rewritten
- Writes are serialized in one process
- No transactional relational model
- Polling rather than realtime subscriptions
- No automated API test suite in package scripts
- Single server file is approximately 2,270 lines

What is stored, and where

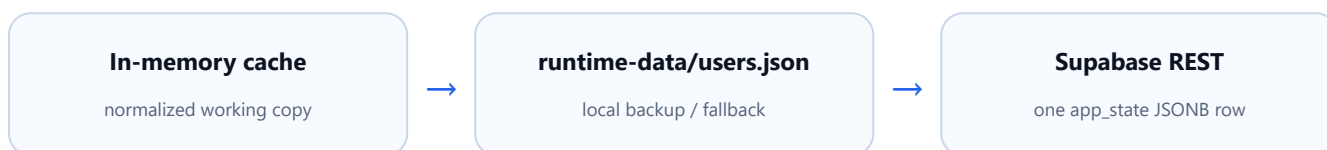
Server-side domains

Users · sessions · communities · direct conversations · duel queue · duels · duel results · generated questions · practice results

Browser-side domains

Session token · theme · practice-library cache · practice progress · analytics events · notification-read IDs

Current persistence model

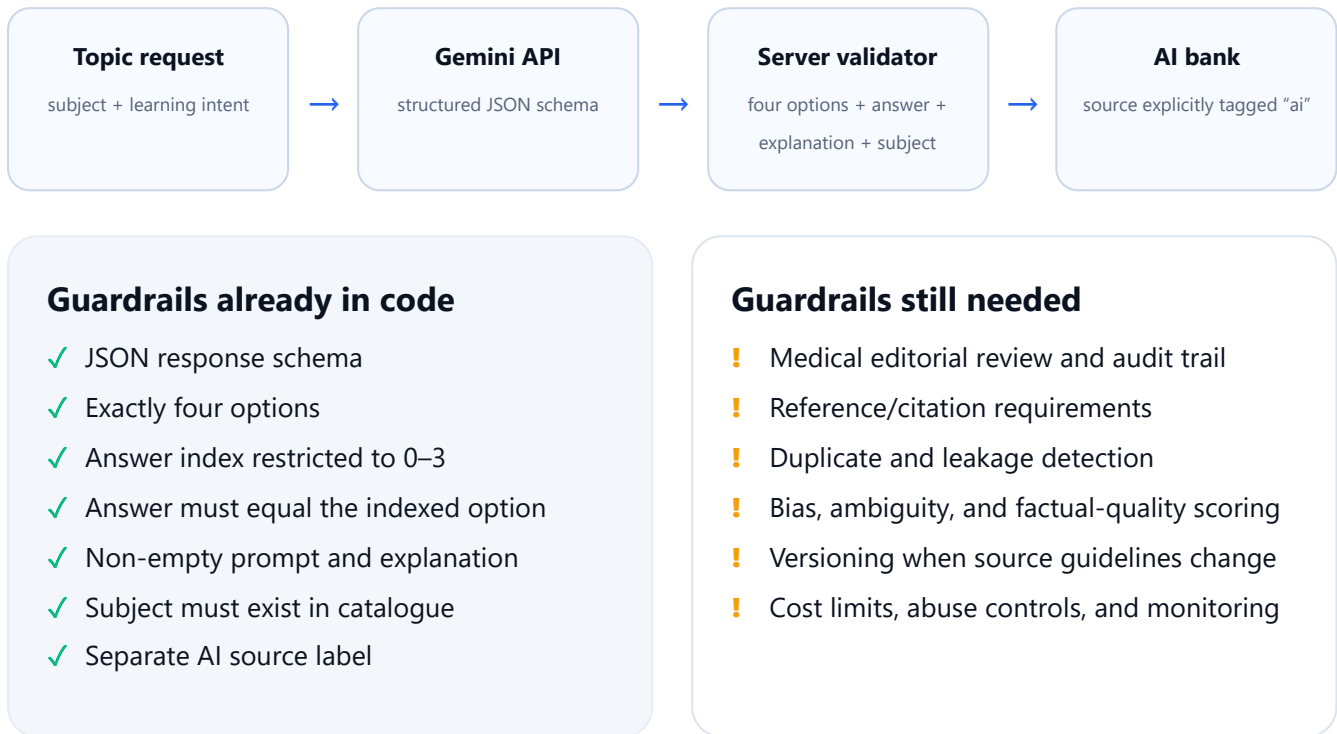


DATA	CURRENT FORM	PRODUCTION EVOLUTION
Users & auth	Custom records + bearer sessions	Managed auth or hardened session service; verification and recovery.
Learning events	Mostly browser-local	Append-only attempts table, cross-device sync, retention policy.
Question bank	Large versioned JSON files	Normalized content tables + search index + editorial workflow.
Community & messages	Arrays inside shared JSON state	Posts/messages tables, indexes, moderation and realtime delivery.
Duels	Queue and result arrays	Transactional match state, idempotency, concurrency guards, realtime events.
Images	Multiple local/static upload folders	Object storage, signed upload policy, scanning, CDN lifecycle.

Why Supabase is currently unusual: row-level security is enabled, but the service stores the entire application snapshot in one service-role-managed JSONB row. Supabase is being used as durable document storage, not yet as a relational product database.

Where artificial intelligence is used

AI is an optional content-generation path for supplemental practice. The configured default model is **Gemini 2.5 Flash**; the app can also run without an AI key.



Pitch-safe wording

"AI expands targeted practice, while deterministic validation and a future clinician-review workflow protect quality. AI questions are labeled separately from source-tagged PYQs."

Avoid saying: "Every question is medically verified" unless the business can provide reviewer identity, process, and evidence for the current inventory.

Current controls and honest gaps

Implemented controls

- ✓ Passwords salted and hashed with PBKDF2-SHA512
- ✓ Timing-safe hash comparison
- ✓ Random bearer-session tokens
- ✓ Public-user response sanitization
- ✓ Profile image size boundary
- ✓ Path basename handling for served uploads
- ✓ Supabase service key kept server-side
- ✓ Row-level security on the state table

Before production scale

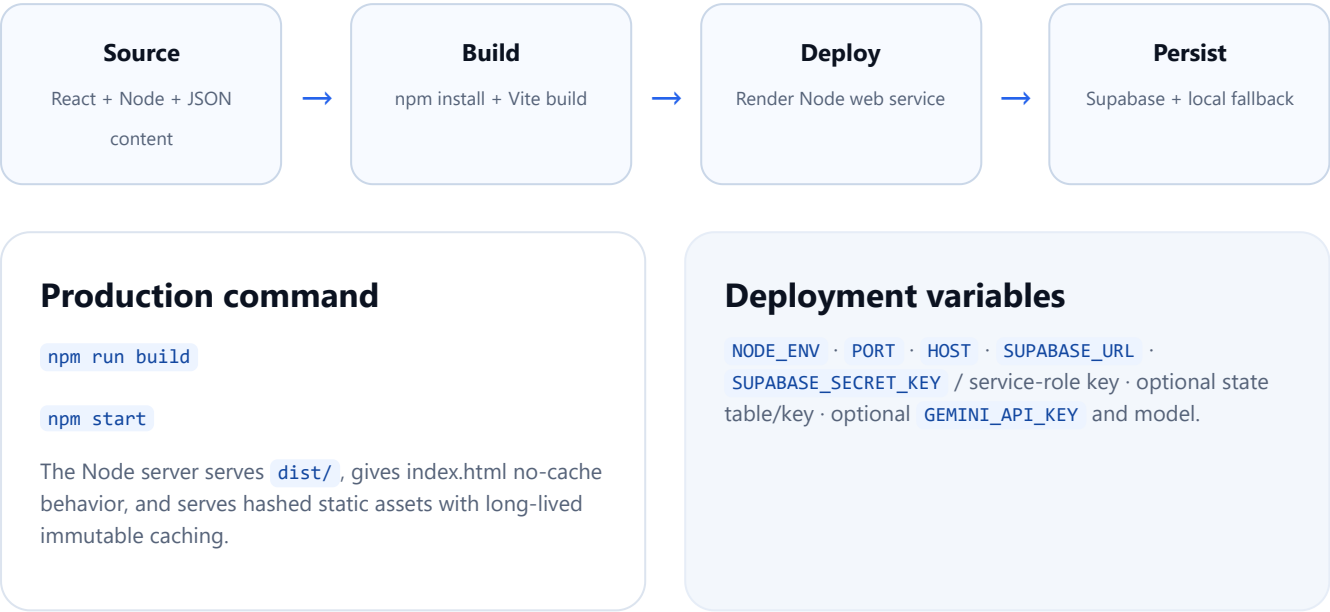
- ! Restrict permissive CORS
- ! Add rate limits and brute-force protection
- ! Add email verification and password recovery
- ! Prefer secure HttpOnly session cookies
- ! Add CSRF strategy if cookies are adopted
- ! Validate image MIME and scan uploads
- ! Add security headers and audit logs
- ! Move secrets to managed deployment storage

Data classification

CLASS	EXAMPLES	RECOMMENDED TREATMENT
Account data	Name, email, phone, medical college	Consent, encryption in transit/at rest, deletion flow, strict access.
Learning data	Attempts, mistakes, pace, weak topics	Treat as private profile data; clear retention and export rules.
Social content	Threads, replies, images, messages	Moderation, reporting, block tools, retention and takedown process.
Payments	Plan, order, entitlement, provider status	Use provider-hosted checkout; never store raw card/UPI credentials.

Positioning boundary: this is an exam-preparation product, not a diagnostic or clinical decision-support system. Terms, disclaimers, and content governance should make that explicit.

How it is built and shipped



Operational readiness checklist

CAPABILITY	TODAY	NEXT OPERATIONAL MILESTONE
Build & serve	CONFIGURED	CI build on every pull request.
Persistent database	FALLBACK MODEL	Normalized Supabase schema and migrations.
Monitoring	NOT EVIDENT	Error tracking, uptime, latency, queue and AI metrics.
Backups	LOCAL COPY	Automated remote backup, restore drill, retention policy.
Automated tests	NO TEST SCRIPT	Unit, API, content-schema, and end-to-end suites.
Release safety	MANUAL	Preview environments, migrations, rollback, feature flags.

Commercial logic already visible in the product

Lite

₹299 / year

Daily practice, analytics, community access, limited battle points.

Ultra

₹999 / year

Unlimited battle points, communities, custom battles, deeper analytics, explanations, selected subjects.

Premium

₹1,499 / month

All subjects, custom modules/practice, exam mode, clinical cases, and Ultra benefits.

These are current interface prices, not proof of activated billing or validated willingness to pay. Checkout is explicitly marked “coming soon.”

Revenue story

Primary

- Freemium student subscriptions
- Subject or exam unlocks
- Annual preparation plans

Expansion options

- College/cohort dashboards
- Faculty/community tools
- Sponsored exam series with strict disclosure

Metrics investors will ask for

ACTIVATION

% completing first 10 questions within 24 hours

RETENTION

D1, D7, D30 learning retention by cohort

LEARNING

Attempts/week, accuracy movement, weak-topic closure

REVENUE

Free-to-paid conversion, ARPU, churn, payback

Best next commercial experiment: test one simple annual plan against a clearly bounded free tier before maintaining three complex entitlements.

A concise story you can deliver

30-second version

"Medical students prepare across disconnected question banks, notes, chats, and score trackers. MediComm brings that loop into one workspace: structured exam practice, clear weak-area analytics, peer communities, and fast 1v1 recall battles. The working product already supports 19 subjects and serves more than 13,000 distinct question records across source-tagged and supplemental modes. Our next step is to validate retention and paid conversion while hardening content review and the data platform."

Five-slide narrative

SLIDE	MESSAGE	EVIDENCE TO SHOW
1. Problem	Preparation is fragmented and feedback is weak.	A student's current tool stack and time lost switching.
2. Product	One learning loop from question to next action.	Live practice → explanation → analytics demo.
3. Differentiation	Content + learning signals + peer network + competition.	Three content modes, weak-topic view, community, duel.
4. Business	Freemium access with focused annual upgrade.	Pricing hypothesis and conversion experiment.
5. Ask	Capital/partnership accelerates validation and production readiness.	90-day milestones and use of funds.

Defensible statements

- ✓ Working React + Node product
- ✓ 19-subject catalogue
- ✓ 15 exam-year collections
- ✓ 13,035 distinct API records after de-duplication
- ✓ Source-separated AI content
- ✓ Profiles, messaging, communities, rankings, and duels
- ✓ Optional Supabase persistence
- ✓ Deployment blueprint included

A seven-minute product walkthrough

1

0:00–0:45

State the fragmentation problem and show the focused sign-in surface.

2

0:45–2:30

Open Practice, choose a subject, solve one strong clinical question.

3

2:30–3:30

Show explanation, confidence, flag, progress, and keyboard speed.

4

3:30–4:30

Open Analytics and connect the attempt to the next action.

5

4:30–5:20

Open a community or direct conversation to show peer context.

6

5:20–6:10

Start a bot duel; explain rated matchmaking without waiting live.

7

6:10–6:40

Show pricing as a hypothesis, not completed commerce.

8

6:40–7:00

Close with the 90-day validation plan and the ask.

Prepare before the room

- ✓ Use a populated demo account
- ✓ Preload one polished question with image/explanation
- ✓ Confirm API and Supabase status
- ✓ Use bot duel for deterministic timing
- ✓ Keep screenshots as offline fallback
- ✓ Disable notifications and browser chrome clutter

Do not demo live

- ! AI generation latency
- ! Waiting for another rated player
- ! Unmoderated community content
- ! Checkout—there is no payment integration
- ! Empty/new-account analytics without seeded history
- ! Any unverified “expert curated” claim

Answer to “what is proprietary?”

“The code stack is standard by design. The defensible layer is the structured medical content graph, the learning-event history, the quality workflow, and the network formed around study behavior. Those assets become stronger as validated usage grows.”

From working MVP to investable system

HORIZON	PRODUCT	TECHNOLOGY & TRUST	PROOF
0–30 days	Polish onboarding and one “first win” practice path.	Fix counter hydration, remove dead UI, add API/content smoke tests.	Instrument activation funnel.
31–60 days	Server-sync attempts and improve weak-topic review.	Normalize users, attempts, questions, messages; harden auth/rate limits.	D1/D7 retention and question completion.
61–90 days	Pilot paid annual plan and reviewed content batch.	Provider-hosted checkout, entitlements, moderation, monitoring.	Conversion, churn intent, qualitative learner outcomes.
3–6 months	Adaptive modules, revision scheduling, cohort tools.	Realtime events, search, object storage, editorial CMS.	Cohort retention and college pilots.

Highest-priority risks

CONTENT PROVENANCE

Confirm rights, source traceability, labeling, and expert review for every claim-worthy content set.



DATA ARCHITECTURE

Replace one-row snapshot storage before meaningful concurrency and analytics volume.



SECURITY & MODERATION

Harden identity, uploads, community safety, logging, and incident response.



MARKET VALIDATION

Prove repeated use and willingness to pay; repository scale is not product-market fit.



Recommended investor ask framing: fund the transition from feature-complete MVP to evidence-backed learning product—quality assurance, learner instrumentation, production data architecture, and a measured paid pilot.

API surface

METHOD	ROUTE	PURPOSE
POST	<code>/api/auth/signup</code>	Create account and session.
POST / GET / POST	<code>/api/auth/login</code> · <code>/session</code> · <code>/logout</code>	Authentication lifecycle.
PATCH	<code>/api/profile</code> · <code>/api/profile/stats</code>	Identity and learning summary.
GET	<code>/api/practice</code>	Assembled practice library; supports source/exam/year/subject/topic filters.
POST	<code>/api/generate-question(s)</code>	Generate and validate one or a batch of supplemental questions.
GET	<code>/api/leaderboard</code> · <code>/api/summary</code>	Ranked users and platform totals.
GET	<code>/api/users/search</code> · <code>/api/users/:id</code>	Peer discovery and public profile.
GET / POST	<code>/api/communities</code>	List and create communities.
POST	<code>/api/communities/:id/join</code>	Join a community.
POST	<code>/api/communities/:id/messages</code>	Publish a thread or reply, optionally with image.
DELETE	<code>/api/communities/:id/members/:userId</code>	Admin member removal.
GET / POST	<code>/api/direct-messages</code> · <code>/open</code>	List or open private conversations.
POST	<code>/api/direct-messages/:id/messages</code>	Send direct text/image message.
GET	<code>/api/duels/questions</code>	Return question set without client answer keys.
POST	<code>/api/duels/complete</code>	Score result and update rating.
POST / GET / DELETE	<code>/api/duels/rated/queue</code>	Join, poll, or leave matchmaking.
GET	<code>/api/storage/status</code>	Inspect Supabase/local persistence health.
GET	<code>/uploads/:file</code>	Serve runtime, built, public, or legacy image asset.

Installed package groups

Runtime: React, React DOM, Radix UI components, Framer Motion, Lucide React, TanStack React Query, React Hook Form, Zod, class-variance-authority, clsx, tailwind-merge. **Build:** Vite React plugin, Tailwind CSS, PostCSS, Autoprefixer, TypeScript. Presence in `package.json` does not imply every library is used by the current screens.

Where everything lives

PATH	RESPONSIBILITY
<code>src/App.jsx</code>	Core screens, feature state, workflows, and client-side analytics.
<code>src/components/AppShellV2.jsx</code>	Desktop/mobile navigation, command menu, notifications, profile menu.
<code>src/styles.css</code> / <code>src/redesign.css</code>	Visual system, layouts, responsive behavior, themes.
<code>src/lib/api.js</code>	Authenticated JSON requests, timeout, and error normalization.
<code>server.mjs</code>	HTTP server, API, auth, persistence, content assembly, AI, community, duels.
<code>data/practice-question-bank.json</code>	Versioned exam catalogue and embedded question records.
<code>runtime-data/users.json</code>	Local runtime database snapshot and supplemental question records.
<code>public/uploads/</code>	Clinical and question image assets (1,146 files at audit time).
<code>scripts/</code>	Import, clean-up, ordering, migration, inventory, and Supabase seed tooling.
<code>supabase/migrations/</code>	App-state table and row-level-security policy.
<code>render.yaml</code>	Render build/start configuration and production variables.
<code>package.json</code>	Node version, scripts, runtime dependencies, and build dependencies.

Due-diligence notes

- ! No automated test script is declared
- ! No payment provider is connected
- ! No evidence of verified active-user counts
- ! Detailed analytics are device-local
- ! Supabase state is not relational yet
- ! CORS currently allows any origin
- ! Content-rights evidence is outside the code
- ! Monitoring and moderation need productization

The strongest pitch is not “everything is finished.” It is: “the complete learning loop is working, the technical and content risks are known, and the next validation milestones are concrete.”

CLOSING THOUGHT

A working foundation. A measurable next step.

Use the product demo to prove the loop. Use the roadmap to prove judgment. Use real cohort data next to prove the business.

Product

Practice + intelligence

Network

Community + competition

Next proof

Retention + paid conversion

